

Advanced Algorithms – Programming Assignment 1

Aaron Falco – 25126703

Table of Contents

Table of Contents.....	1
Section A – What I Built:	1
Section B – Track Specific Write-up.....	2
WhiteFang:	2
Lexer:	2
Parser:	3
Bytecode compiler:	3
Virtual machine	4
Worked Example	5
Step 1: lexing	5
Step 2: parsing.....	6
Step 3: bytecode compilation	6
Step 4: execution	6
Section C – What I Learned:	6
Section D – AI Use:	6

Section A – What I Built:

I built WhiteFang, a small C-family programming language (the name is an inside joke from my D&D group), along with the full pipeline that compiles and runs it: a lexer, a recursive-descent parser, a bytecode compiler, and a virtual machine, all written in C. The original plan was to compile WhiteFang source code to generated C code and build it with gcc/clang, leaning on C's existing optimizer and backend rather than writing a code generator myself. I changed this once I realized that route would teach me the least about the part I actually wanted to understand (how a language actually gets compiled and executed). That decision also meant scaling the language itself down to a simpler language than one aimed at gcc's C front end.

The final architecture is a four-stage pipeline running as a single process with no intermediate files. Source text is tokenized by a lexer, parsed into an abstract syntax tree, compiled from that tree into a custom bytecode format and executed by a stack-based virtual machine. WhiteFang is statically typed with some stylistic changes to syntax (e.g. type names come before variable names). To make the result easy to run, I packaged the whole pipeline as a Docker image (which I wanted to learn more about anyway) with a multi-stage build that compiles the interpreter and ships only the resulting binary, so running a WhiteFang program needs nothing installed locally beyond Docker itself. To navigate the repository: there is one directory per pipeline stage under src/ (lexer/, parser/ plus ast/, bytecode/, vm/), each independently testable through its own small command-line tool. The directory tests/golden/ holds test programs paired with their expected output and docs/ holds the language specification and the full bytecode/VM design.

Section B – Track Specific Write-up

WhiteFang:

In WhiteFang, variables are declared type-first with the initializer required (there's no default or zero value). Functions are declared with the return type trailing the closing brace rather than leading the declaration. Every program's entry point is a function named `start` with its return value becoming the exit code. Four constants (`PI`, `E`, `G`, `R2`) are available everywhere without declaration. They are reserved keywords, not variables, and behave exactly like float literals wherever they're written.

The type-before-name and trailing-return-type forms were both deliberate departures from ordinary C-family syntax as I wanted the language to look and feel different to C. Two features were tried and then removed once their real cost became visible while building the parser: a form that let one let declare several variables at once, and genuine global variables. The multi-name form needed lookahead-dependent parsing logic to tell its two variants apart, for a benefit that only ever amounted to less typing. Global variables existed only to give `_PI/_E/_G/_R2` somewhere to live; once that was made explicit, turning them into reserved keywords instead removed the need for any variable storage for them at all, and, as a side effect, made redeclaring one of them a plain syntax error rather than something needing its own check.

Lexer:

The lexer is a single-pass scanner which, given the raw source text, produces one token at a time and skips over whitespace and comments. Tokens are lightweight where each one is a type tag plus a pointer and length into the original source text, not a copied string, so scanning allocates nothing. Representing a token as a slice of the source rather than an owned string was a deliberate efficiency choice with one real cost: the source buffer has to stay alive for as long as any token derived from it is in use, which holds throughout this project since the whole pipeline runs in one process per program.

A small command-line tool prints every token the lexer produces for a given file, used to manually check tokenization against the language spec while building it. That process caught a real bug: an unterminated block comment was being silently accepted up to the end of the file instead of being reported as an error, later fixed to correctly surface as a lexer error.

Parser:

The parser is a recursive-descent parser that turns the token stream into an abstract syntax tree, using one function per precedence level for expressions (so `a + b * c` is handled by the multiplication-level function being called from inside the addition-level one, mirroring standard precedence directly in the call structure). It looks two tokens ahead at the start of a statement to tell an assignment (`x = 10;`) apart from an expression used as a statement (`PRINT(x);`), since both begin with an identifier. On the first syntax error it reports it and stops immediately, rather than attempting to recover and continue.

Errors were made to stop the parser immediately, rather than attempting to recover and report several errors per run, as a deliberate simplicity choice. Recovering safely after a syntax error was judged as not worth the complexity here. A rule that a variable can't be redeclared within the same block (shadowing a variable from an outer block is still allowed) was added once it became clear the underlying design permitted it for free, and that allowing it by accident was more likely to hide a mistake than express one on purpose.

A second command-line tool prints the parsed tree for a file as indented text, checked by hand against a range of programs: functions with if/else if/else chains, unary and binary expressions, character literals with escape sequences, multi-argument calls, and variables shadowed across nested blocks. The same tool confirmed error cases behave correctly too.

Bytecode compiler:

The compiler walks the parsed tree twice. The first pass records every function's name, parameter types, and return type without compiling any bodies, so a call can be checked regardless of whether the function it targets appears earlier or later in the file. The second pass compiles each function body: resolving every variable name to a local slot, checking that every operation is applied to consistently typed values, and emitting the resulting instructions. Any problem found here is reported as a compile error and, as in the parser, compilation stops at the first one found.

All of this checking was put in the compiler rather than a separate analysis step, since the compiler already needs to know a variable's type and location to generate correct instructions for it. Whether a function can fall off its end without returning was checked with a simple rule (its very last statement must be a return) rather than by tracing every possible path through its if/while branches. While this simple rule does have limitations and occasionally gives false negatives (rejecting a function that would always return in practice) but I decided the tradeoff was worth the time and complexity it saved.

A third tool disassembles the compiled bytecode into readable instruction listings, checked instruction-by-instruction against a hand-worked-out trace of what a small recursive program should compile to, and matched it exactly. Building this stage also surfaced two real gaps in the bytecode format as it had been designed on paper before any code was written: there was no instruction at all for negating a number, and a written rule would have incorrectly allowed arithmetic on boolean and character values. Both were caught and fixed during implementation. Another inefficiency was found on this step as well, as the compiler was emitting a now pointless stack cleanup instruction immediately after a return that had already discarded everything. Beyond the disassembler, a wide range of programs designed specifically to fail were run through the compiler to confirm each produced the intended error: duplicate function names, wrong argument counts and types, a missing return, and attempts to use a reserved keyword as a name.

Virtual machine

The virtual machine is a stack-based interpreter: it holds one large array as its working stack, and a separate, smaller stack of call-frame records for tracking function calls. Values on the stack carry no type information at runtime as the compiler has already resolved every value's type and picked type-specific instructions accordingly, so the machine itself never has to determine the kind of value while running.

Leaving values untagged, rather than pairing every value with a marker recording its type, was possible specifically because WhiteFang is statically typed. Both stacks are a single large fixed size rather than calculated per program, favoring simplicity over precision; a program that recurses too deeply is stopped with a clean error rather than a crash, the same way a program that divides an integer by zero is.

The finished virtual machine was run against the full example-program suite described above and produced the exact expected output in every case, including the target recursive Fibonacci example. Beyond that, it was deliberately pushed on cases designed to fail safely: dividing by zero (confirmed to report a clean error rather than crash), and recursion deep enough to exceed the call-frame limit (confirmed to report a clean "stack overflow" error at 1,000 levels of recursion, while 200 levels completes normally) — both intended to confirm the safety nets built into the design actually hold up rather than just being assumed to.

Worked Example

To make the four-stage pipeline described above concrete, this section traces one real program (the recursive Fibonacci function used throughout this project) through every stage, from the exact source text to the final printed output. This is the WhiteFang source code:

```
func fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n - 1) + fib(n - 2);  
} -> int
```

```
func start() {  
    let int: i = 0;  
    while (i < 10) {  
        PRINT(fib(i));  
        i = i + 1;  
    }  
    return 0;  
} -> int
```

Step 1: lexing

The lexer reduces that text to this exact stream of tokens (line number, token type, the literal text matched). Every keyword, identifier, literal, and piece of punctuation becomes one entry:

1	FUNC	'func'
1	IDENTIFIER	'fib'
1	LPAREN	'('
1	INT	'int'
1	IDENTIFIER	'n'
1	RPAREN	')'
1	LBRACE	'{'
2	IF	'if'
2	LPAREN	'('
2	IDENTIFIER	'n'
2	LESS_EQ	'<='
2	INT_LIT	'1'
2	RPAREN	')'
2	LBRACE	'{'
3	RETURN	'return'
3	IDENTIFIER	'n'
3	SEMICOLON	';'

```
4  RBRACE      '}'
5  RETURN      'return'
5  IDENTIFIER  'fib'
5  LPAREN      '('
5  IDENTIFIER  'n'
5  MINUS       '-'
5  INT_LIT     '1'
5  RPAREN      ')'
5  PLUS        '+'
5  IDENTIFIER  'fib'
5  LPAREN      '('
5  IDENTIFIER  'n'
5  MINUS       '-'
5  INT_LIT     '2'
5  RPAREN      ')'
5  SEMICOLON   ';'
6  RBRACE      '}'
6  ARROW       '->'
6  INT         'int'
8  FUNC        'func'
8  IDENTIFIER  'start'
8  LPAREN      '('
8  RPAREN      ')'
8  LBRACE      '{'
9  LET         'let'
9  INT         'int'
9  COLON       ':'
9  IDENTIFIER  'i'
9  EQ          '='
9  INT_LIT     '0'
9  SEMICOLON   ';'
10 WHILE      'while'
10 LPAREN      '('
10 IDENTIFIER  'i'
10 LESS        '<'
10 INT_LIT     '10'
10 RPAREN      ')'
10 LBRACE      '{'
11 PRINT      'PRINT'
11 LPAREN      '('
11 IDENTIFIER  'fib'
11 LPAREN      '('
11 IDENTIFIER  'i'
11 RPAREN      ')'
11 RPAREN      ')'
11 SEMICOLON   ';'
12 IDENTIFIER  'i'
```

```

12 EQ          '='
12 IDENTIFIER  'i'
12 PLUS        '+'
12 INT_LIT     '1'
12 SEMICOLON   ';'
13 RBRACE      '}'
14 RETURN      'return'
14 INT_LIT     '0'
14 SEMICOLON   ';'
15 RBRACE      '}'
15 ARROW       '->'
15 INT         'int'
16 EOF         ''

```

Step 2: parsing

The parser consumes that token stream and builds an abstract syntax tree, applying the language's precedence rules as it goes (for instance, `fib(n - 1) + fib(n - 2)` becomes an addition node whose two children are call expressions, each containing a subtraction) and resolving `PRINT(...)` directly into its own node rather than an ordinary function call:

Program

```

FuncDecl fib(int n) -> int
  Block
    If
      Binary <=
        Identifier n
        IntLit 1
      Block
        Return
          Identifier n
    Return
      Binary +
        Call fib
          Binary -
            Identifier n
            IntLit 1
        Call fib
          Binary -
            Identifier n
            IntLit 2
FuncDecl start() -> int
  Block
    VarDecl int

```

```

    i =
      IntLit 0
  While
    Binary <
      Identifier i
      IntLit 10
    Block
      ExprStmt
        Print
          Call fib
            Identifier i
      Assign i =
        Binary +
          Identifier i
          IntLit 1
  Return
    IntLit 0

```

Step 3: bytecode compilation

The compiler's first pass records both functions' signatures (fib(int) -> int and start() -> int) before compiling either body, which is what lets fib's own body call fib again. By the time any body is compiled, every function it might call is already known. The second pass then compiles each body in turn; resolving n and i to local stack slots, checking every operation's operand types, and emitting one instruction per operation:

```
== fib (arity 1) ==
```

```
constants:
```

```

[0] as float=1
[1] as float=1
[2] as float=2
0  LOAD_LOCAL    0
2  PUSH_CONST    0
4  ILE
5  JUMP_IF_FALSE +3 -> 11
8  LOAD_LOCAL    0
10 RETURN
11 LOAD_LOCAL    0
13 PUSH_CONST    1
15 ISUB
16 CALL          func=0 argc=1
19 LOAD_LOCAL    0
21 PUSH_CONST    2
23 ISUB
24 CALL          func=0 argc=1
27 IADD
28 RETURN

```

```
== start (arity 0) [start] ==
```

constants:

```
[0] as int=0
[1] as int=10
[2] as int=1
[3] as int=0
0  PUSH_CONST      0
2  LOAD_LOCAL      0
4  PUSH_CONST      1
6  ILT
7  JUMP_IF_FALSE   +16 -> 26
10 LOAD_LOCAL      0
12 CALL            func=0 argc=1
15 PRINT_INT
16 LOAD_LOCAL      0
18 PUSH_CONST      2
20 IADD
21 STORE_LOCAL     0
23 LOOP            -24 -> 2
26 PUSH_CONST      3
28 RETURN
```

Step 4: execution

The virtual machine begins by treating `start` as though it were itself the result of an initial call, so its locals begin at stack position 0. Tracing the very first loop iteration (`i = 0`) in full:

1. `PUSH_CONST 0` pushes 0 — this push is the declaration of `i`, now at stack position 0.
2. The loop condition runs: `LOAD_LOCAL 0` pushes `i` again, `PUSH_CONST 1` pushes 10, `ILT` pops both and pushes 1 (true). `JUMP_IF_FALSE` pops that 1 and, since it's true, does not jump.
3. `LOAD_LOCAL 0` pushes `i` (0) again, as the argument for `fib(i)`. `CALL func=0 argc=1` computes the new call's frame base ($\text{stack_top} - \text{argc} = 2 - 1 = 1$), records where to resume in `start` afterward, and switches execution to `fib`'s chunk.
4. Inside `fib`, with `n = 0` at its local slot: `LOAD_LOCAL 0`, `PUSH_CONST 0` (1), `ILE` — `0 <= 1` is true, so `JUMP_IF_FALSE` does not jump, and `LOAD_LOCAL 0`; `RETURN` returns `n` itself. `RETURN` truncates the stack back to this call's frame base (discarding `n`'s slot) and pushes the return value 0 onto what is now `start`'s stack again, then resumes `start` right after the `CALL`.
5. `PRINT_INT` pops that 0 and prints it.

6. `LOAD_LOCAL 0, PUSH_CONST 2 (1)`, `IADD` computes $i + 1 = 1$; `STORE_LOCAL 0` writes it back into `i`'s existing slot (no new slot — this is why the loop's stack usage doesn't grow with each iteration).
7. `LOOP` jumps back to the start of the condition check, and the cycle repeats with `i = 1`.

Every later iteration follows the identical mechanism; the only thing that changes is the value of `i` and how many nested calls it takes to reach the `n <= 1` base case (each nested call gets its own frame base further up the same shared stack, so `fib(n-1)` and `fib(n-2)` never interfere with each other). Running the compiled program to completion prints the following and returns the status 0:

```
0
1
1
2
3
5
8
13
```

Section C – What I Learned:

The main thing I take away from this project is a concrete, hands-on understanding of one way to actually compile and run a programming language. Not just the theory of what a lexer or a parser is, but the specific mechanics of building one: how a recursive-descent parser turns tokens into a tree, how a compiler resolves names and picks type-specific instructions, how a stack-based virtual machine executes those instructions one at a time. Before this project I could describe those stages in the abstract; now I've written all four of them.

In terms of things that surprised me or that I didn't expect, I underestimated the project's complexity going in. It showed up most clearly in scope I had to cut before I'd written a line of parser code: I originally expected to add arrays and for loops as I went and realized fairly quickly that both were unrealistic within the project's timeframe, pushing them onto a stretch-goal list I never got back to. I also changed course on the compilation strategy itself early on. My first plan was to transpile WhiteFang source into C and let gcc or clang handle the rest, but I decided against that once I realized it would hand off exactly the parts of the process I actually wanted to build myself. A smaller, more concrete example of the same underestimation showed up directly in the language's grammar: I originally allowed

declaring several variables in one `let` statement, but once I got to writing the actual parsing logic for it I found it needed real lookahead-based parsing to tell its two forms apart which would have resulted in a real increase in complexity for a feature that only ever saved a little typing.

What I understand now that I didn't before starting is how fast complexity compounds from features that sound small in isolation. The clearest example was error reporting: a compiler that reports every error it finds in one pass, instead of stopping at the first one, sounds like obviously nicer behavior. However, actually building it would have meant a substantial amount of extra machinery just to recover safely from one mistake before looking for the next, so I kept the simpler stop-on-first-error behavior instead. I also came away with a much stronger appreciation for the importance of code readability than I had going in, having spent real time re-reading my own compiler and VM code well after writing it and being grateful, in hindsight, for every place I'd bothered to write it clearly the first time.

Two things genuinely surprised me. The first was how hard it actually is to find a piece of syntax that's distinctive, doesn't look bad, and doesn't break parsing. Settling on WhiteFang's comment syntax took several failed attempts, because most punctuation characters are either extremely common in ordinary code already or already claimed by some other language's comments. The second was how unevenly complexity was distributed across features that seemed similarly sized going in: recursion, which I expected to be one of the harder things to get right in the virtual machine, turned out to fall out almost for free from the way function calls and local variables were already designed, while reliably checking that every function actually returns a value on every possible path turned out to be complex enough that I only implemented a simplified version of that check, and left the fuller version as a known limitation rather than build it.

Section D – AI Use:

Claude was used as the main AI tool throughout the project. It was involved at every stage of development, including initial scaffolding, implementation, code generation, and explaining relevant concepts. This made Claude a consistent part of the development process rather than a tool used only for isolated tasks.

However, Claude was not always correct or helpful. One example was the initial recommendation to compile the project using clang. After conducting further research independently, I decided to use the Bytecode C Compiler (BCC) instead, as this provided a better opportunity to learn about the compilation process itself. This demonstrated that

Claude's recommendations needed to be evaluated against independent research, particularly when the choice of approach affected the project's learning objectives.

A second issue occurred during the design process, when Claude repeatedly executed a shell command that had no useful purpose. It became stuck in a loop, calling the command multiple times within a single session, and I had to manually stop the process. This was an example of AI-generated behavior that was neither productive nor self-correcting. There were also some unnecessary additions to the project that were not identified by Claude, reinforcing the need for human oversight.

At the outset of the project, I had a general understanding of how code compilation works. I understood the overall purpose of the compilation pipeline and the intent behind its individual stages, including what each stage was attempting to achieve. This existing knowledge allowed me to understand the concepts being discussed and to recognize when further investigation was necessary.

In contrast, I relied heavily on Claude for code generation. Rather than independently deriving every implementation detail, I generally trusted the generated code and validated it primarily by checking its output and functionality. This meant that my understanding of the underlying concepts was stronger than my understanding of every implementation detail in the generated code. The project therefore involved a combination of existing conceptual knowledge, AI-assisted implementation, and independent validation where Claude's recommendations or behavior appeared questionable.